FF-AODV Implementation Guide

Energy and Mobility-Aware Routing for Disaster Response Drone Networks (FANETs)

Step-by-Step Implementation Guidelines for NS-3

Based on Project Proposal and Research Papers:

[1] Taha et al. - Energy Efficient Multipath Routing Protocol (FF-AOMDV)
[2] Yang & Liu - Optimization Routing Protocol for FANETs

Table of Contents

1. Project Overview

This implementation guide provides a comprehensive roadmap for developing an Energy and Mobility-Aware Routing Protocol for Flying Ad-Hoc Networks (FANETs) using the NS-3 simulator. The project addresses a critical challenge in disaster response scenarios: maintaining reliable communication among UAVs (drones) while optimizing energy consumption and accounting for high-speed node mobility.

1.1 Problem Statement

In disaster response scenarios, when ground infrastructure is destroyed, a swarm of Unmanned Aerial Vehicles (UAVs) can create a Flying Ad-Hoc Network (FANET) to relay live video footage from search areas to a remote Base Station. However, standard routing protocols like AODV (Ad-hoc On-Demand Distance Vector) use hop count as the sole metric for route selection, leading to several critical problems that can cause mission failure.

The fundamental flaw in standard AODV routing is that it blindly selects the shortest path without considering node energy levels or mobility patterns. This approach results in three major consequences: first, critical nodes die prematurely due to energy depletion as traffic concentrates on central nodes; second, fast-moving nodes break links immediately causing route instability; and third, high packet loss occurs during frequent route reconstructions. These issues are particularly severe in FANETs where nodes move at high speeds (20-50 m/s) and operate on limited battery power.

1.2 Project Objectives

The project is divided into two phases with distinct objectives. Phase 1 focuses on reproducing the Fitness Function AODV (FF-AODV) methodology proposed by Taha et al. This phase replaces the shortest path logic with a multi-metric fitness function that calculates a score combining residual energy and distance. The goal is to achieve load balancing by routing traffic away from low-battery nodes.

Phase 2 introduces a custom modification called Velocity-Aware Optimization. This extension addresses the limitation of Taha et al.'s approach, which does not account for node speed. By extending the fitness function to penalize high-velocity nodes, the protocol prevents selecting unstable links that would break quickly due to node movement. This is particularly important for FANETs where high-speed mobility is common.

2. Theoretical Background

2.1 AODV Routing Protocol Basics

The Ad-hoc On-Demand Distance Vector (AODV) protocol is a reactive routing protocol designed for mobile ad-hoc networks. It operates on an on-demand basis, meaning routes are established only when needed for data transmission. This approach reduces routing overhead compared to proactive protocols that continuously maintain routing tables for all possible destinations.

AODV uses three types of control messages: Route Request (RREQ), Route Reply (RREP), and Route Error (RERR). When a source node needs to send data to a destination for which it has no valid route, it broadcasts a RREQ packet. Intermediate nodes that receive the RREQ create a reverse route entry and forward the packet.

When the RREQ reaches either the destination or an intermediate node with a valid route to the destination, a RREP is sent back along the reverse path.

The standard AODV route selection criterion is based solely on the minimum hop count. When a node receives multiple RREQ packets for the same destination, it updates its routing table only if the new route has fewer hops. This decision-making process ignores critical factors such as node energy levels and mobility patterns, which can lead to suboptimal route selection in resource-constrained and highly dynamic environments like FANETs.

2.2 Fitness Function Concept

The fitness function is an optimization technique used to evaluate and compare different route options based on multiple criteria. In the context of FANET routing, the fitness function provides a numerical score for each potential route, allowing the protocol to select paths that balance energy efficiency with other performance metrics rather than simply choosing the shortest path.

The base fitness function formula proposed by Taha et al. is designed to maximize route quality by considering two key parameters: residual energy and hop count. The formula is expressed as:

$$F = \text{alpha} \times (E_{residual} / E_{initial}) + \text{beta} \times (1 / \text{HopCount})$$

In this equation, alpha (typically 0.6) represents the weight assigned to the energy component, while beta (typically 0.4) represents the weight assigned to the distance component. The term E_residual/E_initial represents the normalized residual energy ratio, ensuring that routes with higher remaining energy are preferred. The term 1/HopCount ensures that shorter routes are favored when energy levels are comparable.

The algorithmic change from standard AODV is straightforward but impactful. Instead of updating a route when Hops_new < Hops_old, FF-AODV updates a route when F_new > F_old. This modification ensures that routes with better energy characteristics are selected even if they have more hops, promoting load balancing across the network and extending the lifetime of individual nodes.

2.3 Velocity-Aware Optimization

The velocity-aware extension addresses a critical gap in the base fitness function: it does not consider the mobility patterns of nodes. In FANETs, nodes move at high speeds (typically 20-50 m/s), and selecting a route that includes fast-moving nodes leads to frequent link failures. This extension adds a third parameter to the fitness function to penalize high-velocity nodes.

The velocity-aware fitness function introduces a normalized velocity factor calculated as:

$$F_{new} = F_{base} + \text{gamma} \times ((V_{max} - V_{current}) / V_{max})$$

In this formula, gamma (typically 0.2) represents the weight assigned to the stability component. V_max represents the maximum expected velocity in the network, while V_current represents the current velocity of the node being evaluated. When a node is hovering (V_current = 0), the stability score is maximized at 1.0, indicating the most stable condition. As the node's velocity increases, the stability score decreases proportionally.

The integration logic for velocity-aware routing follows a four-step process. First, when a RREQ arrives at a node, the node checks its current speed against the maximum expected speed. Second, the node calculates a stability score based on its normalized velocity. Third, this stability score is incorporated into the overall fitness calculation. Fourth, the routing table is updated only if the new path offers the best balance of energy, distance, and stability. This comprehensive approach ensures that routes are selected based on multiple criteria that directly impact network performance.

3. NS-3 Architecture Overview

Understanding the NS-3 architecture is essential for implementing the fitness function modifications. NS-3 is a discrete-event network simulator that provides models for various network protocols, devices, and scenarios. The AODV implementation in NS-3 is located in the src/aodv directory and consists of several key files that need to be modified for this project.

Key NS-3 AODV Files:

File Name	Purpose	Modifications Required
aodv-routing-protocol.cc	Main routing protocol implementation	Add fitness calculation logic in RecvRequest()
aodv-routing-protocol.h	Header file for routing protocol	Add member variables for fitness
aodv-packet.h	RREQ/RREP packet definitions	Add m_pathFitness field
aodv-packet.cc	Packet serialization implementation	Implement fitness field serialization
aodv-rtable.cc	Routing table implementation	Add fitness field to entries
aodv-rtable.h	Routing table header	Define fitness field structure

The NS-3 simulator follows a modular architecture where each network component is implemented as a separate class. The Node class represents a basic computing device, while NetDevice classes represent network interface cards. Applications run on nodes to generate and consume traffic, and Channels represent the communication medium. Helper classes simplify the configuration and interconnection of these components.

For energy modeling, NS-3 provides the Energy module (src/energy) which includes BasicEnergySource for modeling battery capacity and DeviceEnergyModel for tracking energy consumption by network devices. The Mobility module (src/mobility) provides various mobility models including the Gauss-Markov model recommended for FANET simulations, as well as methods to query node velocity through MobilityModel::GetVelocity().

4. Implementation Roadmap

The implementation follows a structured approach divided into distinct phases. This roadmap provides a high-level overview of the development process, from initial setup through testing and evaluation.

Phase	Task	Files to Modify	Key Activities
Setup	Environment Setup	scratch/	Install NS-3.39, configure build system
Phase 1	FF-AODV Core	src/aodv/*	Add fitness fields, implement calculation
Phase 1	Energy Setup	scratch/	Install BasicEnergySource on nodes
Phase 2	Velocity Extension	src/aodv/*	Add velocity tracking, extend fitness
Testing	Simulation	scratch/	Create simulation script, run tests
Analysis	Evaluation	scratch/	Calculate metrics, generate plots

Before beginning the implementation, ensure that NS-3 version 3.39 is properly installed and the basic examples run correctly. Test the installation by running './ns3 run hello-simulator' and verifying that it outputs 'Hello Simulator'. Also test the AODV example located in examples/routing/ to understand the basic AODV simulation setup.

5. Phase 1: FF-AODV Implementation

5.1 Files to Modify

The core implementation of FF-AODV requires modifications to several files in the src/aodv directory. The primary file for routing logic is aodv-routing-protocol.cc, which contains the RecvRequest() method where route selection decisions are made. The packet definition files (aodv-packet.h and aodv-packet.cc) need modifications to carry fitness information in RREQ packets.

Start by examining the existing AODV implementation to understand the code structure. The key methods to understand are RecvRequest() for handling incoming RREQ packets, RecvReply() for handling RREP packets, and the routing table operations in aodv-rtable.cc. These methods form the foundation for the fitness function modifications.

Step-by-Step File Modifications:

Step 1: Modify aodv-packet.h - Add a new member variable to the RREQ header to store the path fitness value. This field will accumulate fitness information as the RREQ travels through the network. The declaration should be added in the public section of the RreqHeader class. Add: 'double m_pathFitness;' to store the accumulated fitness score. Also add getter and setter methods: 'void SetPathFitness(double fitness);' and 'double GetPathFitness() const;'.

Step 2: Modify aodv-packet.cc - Implement the serialization and deserialization of the new fitness field. In the Serialize() method, add code to write the fitness value to the buffer. In the Deserialize() method, add code to read the fitness value. The GetSerializedSize() method must also be updated to account for the additional bytes (sizeof(double) = 8 bytes).

Step 3: Modify aodv-rtable.h - Add a fitness field to the RoutingTableEntry class. This stores the fitness value for each route in the table. Add: 'double m_fitness;' as a member variable, along with corresponding getter and setter methods.

5.2 Code Changes Details

The most significant code changes occur in aodv-routing-protocol.cc, specifically in the RecvRequest() method. This method is called when a RREQ packet arrives at a node, and it is where the routing decision is made. The standard AODV logic updates the route if the new hop count is lower; FF-AODV modifies this to update based on fitness score.

The fitness calculation requires access to the node's residual energy. In NS-3, this is obtained through the Energy module. The node must have a BasicEnergySource installed, and the code must retrieve this energy source to query the remaining energy. The following pseudocode illustrates the modification to RecvRequest():

```
// In RecvRequest() method - after receiving RREQ:
// 1. Get node's residual energy
Ptr energySource =
m_ipv4->GetObject();
double residualEnergy = energySource->GetRemainingEnergy();

// 2. Get initial energy (constant for simulation)
double initialEnergy = 100.0; // Joules (from simulation setup)

// 3. Calculate energy component
double energyRatio = residualEnergy / initialEnergy;

// 4. Calculate hop count component
uint8_t hopCount = rreqHeader.GetHopCount() + 1;
double hopComponent = 1.0 / hopCount;

// 5. Calculate fitness score
double alpha = 0.6; // Energy weight
double beta = 0.4; // Distance weight
double fitness = alpha * energyRatio + beta * hopComponent;

// 6. Update RREQ with accumulated fitness
rreqHeader.SetPathFitness(fitness);

// 7. Route update decision (replace hop count check)
if (fitness > storedFitness) {
// Update route with new fitness
routingTableEntry.SetFitness(fitness);
}
```

5.3 Energy Source Setup

The energy source must be configured in the simulation script (typically a .cc file in the scratch/ directory). NS-3 provides the BasicEnergySource class to model battery capacity and the WifiRadioEnergyModel to track energy consumption by WiFi interfaces. The setup requires creating energy sources for each node and installing them before the simulation begins.

The energy configuration code should be added to the simulation script after node creation. The BasicEnergySourceHelper simplifies the setup process by providing methods to configure initial energy and install energy sources on nodes. The following code demonstrates the energy setup:

```
// Energy source configuration in simulation script
BasicEnergySourceHelper energySourceHelper;
energySourceHelper.Set("BasicEnergySourceInitialEnergyJ",
DoubleValue(100.0)); // 100 Joules initial

// Install energy sources on all nodes
EnergySourceContainer energySources =
energySourceHelper.Install(nodes);

// Create device energy model
WifiRadioEnergyModelHelper radioEnergyHelper;
radioEnergyHelper.Set("TxCurrentA", DoubleValue(0.0174)); // Transmission current
radioEnergyHelper.Set("RxCurrentA", DoubleValue(0.0197)); // Reception current

// Install on WiFi devices
DeviceEnergyModelContainer deviceModels =
radioEnergyHelper.Install(wifiDevices, energySources);
```

6. Phase 2: Velocity-Aware Extension

The velocity-aware extension builds upon the Phase 1 implementation by adding a third parameter to the fitness function. This extension requires accessing node mobility information through the MobilityModel interface. The GetVelocity() method returns a Vector containing the current velocity components, and the GetNorm() method calculates the magnitude of the velocity vector.

6.1 Mobility Model Access

Each node must have a MobilityModel installed to enable velocity tracking. For FANET simulations, the Gauss-Markov mobility model is recommended as it provides realistic drone movement patterns with correlation between successive velocity vectors. The following code shows how to access velocity:

```
// Access node velocity in RecvRequest()
Ptr mobility =
m_ipv4->GetObject();

if (mobility) {
Vector velocity = mobility->GetVelocity();
double currentSpeed = velocity.GetNorm(); // m/s

// Maximum expected speed for FANET
double maxSpeed = 50.0; // m/s

// Calculate stability score
double stabilityScore = (maxSpeed - currentSpeed) / maxSpeed;

// Add to fitness (gamma = 0.2)
```

```
double gamma = 0.2;
double extendedFitness = baseFitness + gamma * stabilityScore;
}
```

6.2 Complete Velocity-Aware Fitness Function

The complete velocity-aware fitness function combines all three parameters: energy, distance, and stability. The weights (alpha, beta, gamma) should sum to 1.0 for normalization. The recommended values from the project proposal are alpha = 0.5, beta = 0.3, and gamma = 0.2, giving the highest priority to energy efficiency while still considering distance and stability.

```
// Complete velocity-aware fitness calculation
double CalculateFitness(double residualEnergy, double initialEnergy,
uint8_t hopCount, double currentSpeed,
double maxSpeed) {
// Weight parameters
double alpha = 0.5; // Energy weight
double beta = 0.3; // Distance weight
double gamma = 0.2; // Stability weight

// Energy component (normalized)
double energyComponent = residualEnergy / initialEnergy;

// Distance component (inverse hop count)
double distanceComponent = 1.0 / hopCount;

// Stability component (normalized velocity)
double stabilityComponent = (maxSpeed - currentSpeed) / maxSpeed;

// Combined fitness score
double fitness = alpha * energyComponent +
beta * distanceComponent +
gamma * stabilityComponent;

return fitness;
}
```

6.3 Packet Header Extension for Velocity

To carry velocity information along the RREQ path, the packet header must be extended with an additional field. This allows intermediate nodes to accumulate velocity information as the RREQ travels through the network. The approach can be either storing the maximum velocity encountered or calculating an average velocity along the path.

Add to aodv-packet.h: 'double m_maxVelocity;' to track the maximum velocity encountered on the path. Also add 'double m_velocitySum;' and 'uint8_t m_velocityCount;' for average velocity calculation if desired. Implement corresponding getter/setter methods in aodv-packet.cc along with serialization.

7. Simulation Setup

The simulation setup requires creating a comprehensive script that configures all network components, installs the modified AODV protocol, and runs the experiment with appropriate data collection. The script should be placed in the scratch/ directory and compiled using './ns3 build'.

7.1 Simulation Parameters

Parameter	Value	Unit	Description
Simulator	NS-3.39	-	Network simulator version
Simulation Area	1000 x 1000	meters	Disaster zone area
Number of Nodes	30	UAVs	Drone count in network
Mobility Model	Gauss-Markov	-	Random flight pattern
Node Speed	0 - 50	m/s	Velocity range
Traffic Type	UDP/CBR	-	Video stream simulation
Initial Energy	100	Joules	Battery capacity
Simulation Time	100	seconds	Experiment duration
Alpha (energy)	0.5	-	Energy weight
Beta (distance)	0.3	-	Hop count weight
Gamma (stability)	0.2	-	Velocity weight

7.2 Simulation Script Structure

The simulation script should follow a standard NS-3 structure with the following components: command line argument parsing for parameter variation, node creation, mobility model installation, WiFi channel and device configuration, protocol stack installation, AODV routing configuration, energy source installation, application setup (CBR/UDP for video traffic), tracing configuration, and simulation execution.

```
// Basic simulation script structure
int main(int argc, char *argv[]) {
// 1. Command line arguments
uint32_t nNodes = 30;
double simTime = 100.0;
CommandLine cmd;
cmd.AddValue("nNodes", "Number of nodes", nNodes);
cmd.AddValue("simTime", "Simulation time", simTime);
cmd.Parse(argc, argv);

// 2. Create nodes
NodeContainer nodes;
nodes.Create(nNodes);

// 3. Install mobility model
MobilityHelper mobility;
mobility.SetMobilityModel("ns3::GaussMarkovMobilityModel",
"MeanVelocity", DoubleValue(25.0),
"Alpha", DoubleValue(0.5));
```

```
mobility.Install(nodes);

// 4. Configure WiFi
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211a);
YansWifiPhyHelper wifiPhy;
YansWifiChannelHelper wifiChannel;
wifiChannel.AddPropagationLoss("ns3::RangePropagationLossModel");
wifiPhy.SetChannel(wifiChannel.Create());

// 5. Install Internet stack with AODV
AodvHelper aodv;
InternetStackHelper internet;
internet.SetRoutingHelper(aodv);
internet.Install(nodes);

// 6. Install energy sources
BasicEnergySourceHelper energyHelper;
energyHelper.Set("BasicEnergySourceInitialEnergyJ",
DoubleValue(100.0));
EnergySourceContainer sources = energyHelper.Install(nodes);

// 7. Setup traffic applications
// ... (CBR source and sink setup)

// 8. Enable tracing
wifiPhy.EnablePcapAll("ff-aodv");

// 9. Run simulation
Simulator::Stop(Seconds(simTime));
Simulator::Run();
Simulator::Destroy();

return 0;
}
```

8. Performance Metrics

The evaluation of FF-AODV against standard AODV requires measuring several key performance indicators. These metrics capture different aspects of network performance and energy efficiency, allowing comprehensive comparison between the protocols.

8.1 Key Metrics

Metric	Formula	Description
Network Lifetime	Time until first node death	Measures energy efficiency of routing
Packet Delivery Ratio	PDR = Packets_received / Packets_sent	Measures reliability of data delivery
Average Throughput	Throughput = Bytes_received / Time	Measures network capacity utilization

End-to-End Delay	Delay = Avg(Receive_time - Send_time)	Measures latency performance
Energy Consumption	EC = Sum(Initial - Remaining)	Total energy consumed by network
Routing Overhead	RO = Control_packets / Data_packets	Measures protocol efficiency

8.2 Expected Results

Based on the research by Taha et al. and the project hypothesis, FF-AODV with velocity awareness should demonstrate significant improvements over standard AODV. The expected outcomes include: increased network lifetime due to load balancing across nodes with higher energy; improved packet delivery ratio by avoiding fast-moving nodes that cause link failures; and maintained throughput through more stable route selections. The velocity-aware extension should particularly improve performance in high-mobility scenarios (20-50 m/s).

The research by Taha et al. showed that FF-AOMDV achieved approximately 15-20% improvement in network lifetime compared to standard AOMDV. For FANET scenarios with the velocity-aware extension, even greater improvements in packet delivery ratio are expected during high-speed movement conditions due to the avoidance of unstable links.

9. Troubleshooting Guide

During implementation, several common issues may arise. This section provides guidance for resolving frequently encountered problems.

9.1 Build Errors

Undefined reference to fitness methods: Ensure that all new methods in aodv-packet.cc and aodv-rtable.cc are properly declared in the corresponding header files. The method signatures must match exactly between declaration and definition. Run './ns3 clean' followed by './ns3 build' to ensure all dependencies are rebuilt.

Energy source not found: Verify that the Energy module is enabled in the build configuration. Check that the header #include "ns3/basic-energy-source.h" is present and that the energy source is installed on nodes before the simulation starts. The BasicEnergySourceHelper must be used to install energy sources on the node container.

9.2 Runtime Issues

Segmentation fault in RecvRequest(): This typically occurs when accessing energy or mobility models that have not been installed. Add null pointer checks before accessing these objects. Use Ptr energy = node->GetObject(); and verify that energy is not null before calling GetRemainingEnergy().

Routes not updating based on fitness: Verify that the fitness comparison logic is implemented correctly. The standard AODV checks if (hopCount < existingHopCount), but FF-AODV should check if (newFitness > existingFitness). Also ensure that the fitness value is being correctly propagated through the RREQ packet as it

travels through the network.

9.3 Debugging Tips

Use NS-3's logging system to trace execution flow. Enable logging for AODV with: export NS_LOG='AodvRoutingProtocol=level_all|prefix_func'. Add custom log messages using NS_LOG_LOGIC() and NS_LOG_DEBUG() macros at key decision points. For energy-related debugging, enable logging for the Energy module: export NS_LOG='BasicEnergySource=level_all'.

10. References

[1] A. Taha, R. Alsaqour, M. Uddin, M. Abdelhaq and T. Saba, "Energy Efficient Multipath Routing Protocol for Mobile Ad-Hoc Network Using the Fitness Function," IEEE Access, vol. 5, pp. 10369-10381, 2017. DOI: 10.1109/ACCESS.2017.2707537

[2] H. Yang and Z. Liu, "An optimization routing protocol for FANETs," EURASIP Journal on Wireless Communications and Networking, vol. 2019, no. 120, 2019. DOI: 10.1186/s13638-019-1442-0

[3] ns-3 Consortium, "ns-3 Network Simulator," https://www.nsnam.org/

[4] ns-3 Tutorial, https://www.nsnam.org/docs/release/3.39/tutorial/ns-3-tutorial.pdf

[5] ns-3 AODV Model Documentation, https://www.nsnam.org/docs/release/3.39/models/html/aodv.html

[6] ns-3 Energy Model Documentation, https://www.nsnam.org/docs/release/3.39/models/html/energy.html

[7] ns-3 Mobility Model Documentation, https://www.nsnam.org/docs/release/3.39/models/html/mobility.html